Sparrowi RAG Chatbot

Complete File System & Architecture Documentation

Project: Sparrow Interactive AI Sales Assistant

Location: C:\Users\User\Desktop\Chatbot\

Total Size: 1.5 GB | 92 Qdrant Chunks | Python + React + Node.js

Generated: March 2026

1. Project Overview

This project is an AI-powered RAG (Retrieval-Augmented Generation) sales chatbot built for Sparrow Interactive Pvt Ltd, a leading experiential technology and phygital solutions company specializing in real estate. The chatbot is embedded on the company's website (sparrowi.com) and also available via WhatsApp. It answers customer questions using company knowledge retrieved from a vector database (Qdrant), processed through a re-ranker, and answered by the Sarvam AI LLM.

Technology Stack

Component	Technology
Backend Framework	FastAPI (Python) running on Uvicorn, port 8003
LLM / AI Model	Sarvam AI (sarvam-m model) via sarvamai SDK
Embedding Model	all-MiniLM-L6-v2 (sentence-transformers) - 384 dimensions
Re-ranker Model	cross-encoder/ms-marco-MiniLM-L-6-v2
Vector Database	Qdrant (local binary qdrant.exe) on port 6333
Relational Database	SQLite (sparrowi.db) - leads, conversations, analytics
Session Storage	Redis (optional, falls back to in-memory OrderedDict)
Frontend Widget	Vanilla HTML/CSS/JS chatbot widget injected into website
Admin Dashboard	React + Vite + TailwindCSS + shadcn/ui
WhatsApp Bridge	Node.js + @whiskeysockets/baileys on port 8004
PDF Parsing	pdfplumber + PyPDF2 + Tesseract OCR (PyMuPDF)
Web Scraping	httpx + BeautifulSoup4
Email System	SMTP (Gmail) + Microsoft Graph API + Jinja2 templates

2. Top-Level Directory Structure

```
C:\Users\User\Desktop\Chatbot\          (1.5 GB total)
|-- Chatbot/                238 MB   Scraped mirror of www.sparrowi.com (HTTrack)
|-- rag-chatbot/            1.3 GB   The main application
|-- .claude/                          Claude Code project memory
|-- index.html                        HTTrack mirror index page
|-- backblue.gif, fade.gif            HTTrack UI assets
+-- Chatbot.whtt                      HTTrack project file
```

rag-chatbot/ Directory (Main Application)

```
rag-chatbot/                          (1.3 GB)
|-- backend/           1 MB     Python backend (FastAPI) - all server logic
|-- frontend/          124 KB   Chatbot widget UI files
|-- admin-dashboard/   140 MB   React admin panel (includes node_modules)
|-- whatsapp-bridge/   54 MB    WhatsApp integration (Node.js)
|-- data/              17 MB    SQLite database
|-- venv/              962 MB   Python virtual environment
|-- storage/           2 KB     Qdrant on-disk storage (EMPTY collections)
|-- snapshots/         0        Qdrant snapshots (empty)
|-- qdrant.exe         70 MB    Qdrant vector database binary
|-- .env                        API keys and configuration
|-- run.py                      Startup script
|-- requirements.txt            Python dependencies
|-- test_*.py (many)            Test suites
+-- .qdrant-initialized         Setup flag file
```

3. Backend Directory (backend/)

The backend is the brain of the application. All Python source files live here.

main.py (60 KB)

FastAPI application with ALL API endpoints:
- /chat - Main chat endpoint (receives user message, returns AI response)
- /ingest - Upload SQL/PDF files for ingestion into Qdrant
- /scrape - Trigger live website scraping
- /health - Health check endpoint
- /admin/* - Admin dashboard API (leads, analytics, settings)
- /api/leads/* - Lead management CRUD
- /api/email/* - Email campaign management
- Serves the scraped website mirror with chatbot widget injected into every page

rag_pipeline.py (104 KB)

Core RAG orchestration - the largest and most important file:
1. Receives user query
2. Checks for greetings/small-talk (instant response, no API call)
3. Embeds query using MiniLM
4. Vector search in Qdrant (cosine similarity)
5. Keyword search in Qdrant (IDF-weighted)
6. Merges and deduplicates results
7. Re-ranks with cross-encoder
8. Builds context from top chunks
9. Calls Sarvam AI with context + conversation history
10. Returns response with sources and follow-up suggestions
Also handles: response caching, grounding verification, conversation memory

config.py (2.6 KB)

Loads .env file and exposes all settings via Pydantic BaseSettings:
Sarvam AI key, Qdrant host/port/collection, embedding model, server host/port,
RAG parameters (top_k=6, min_score=0.35), reranker settings, scraper config,
Redis config, SQLite path, SMTP/Microsoft Graph email settings, lead scoring thresholds

claude_client.py (3.9 KB)

Wrapper around Sarvam AI SDK (not Claude despite the filename).
- ask_with_history(): Sends messages with conversation history (last 4 turns)
- ask(): Single message shorthand
- Timeout handling: 15s for answers, 8s for short decision calls
- Strips <think> tags from model output

embeddings.py (1 KB)

Sentence-transformers wrapper:
- Lazy-loads all-MiniLM-L6-v2 model on first use
- embed_text(): Single text embedding (cached with LRU, max 4096)
- embed_batch(): Batch embedding for ingestion (batch_size=64)
- Output: 384-dimensional normalized vectors

vector_store.py (10.4 KB)

Qdrant client with all database operations:
- ensure_collection(): Creates 'sql_chunks' collection if missing
- upsert_chunks(): Inserts vectors in batches of 100
- search(): Cosine similarity search with score filtering and dedup
- keyword_search(): Full-text keyword search with IDF weighting and 10-min cache
- reset_collection(): Clears all data
- delete_by_table(): Deletes chunks by source table name

reranker.py (1.9 KB)

Cross-encoder re-ranker for precision:
- Lazy-loads cross-encoder/ms-marco-MiniLM-L-6-v2
- Scores (query, chunk) pairs with the cross-encoder
- Filters results below min score threshold (-5.0)
- Returns top_k=3 most relevant chunks

chunker.py (8.8 KB)

Text chunking engine:
- Converts SQL tables, PDF pages, and website text into Chunk objects
- Max 1500 chars per chunk, min 60 chars (skip near-empty)
- Splits by paragraph breaks, then newlines, then sentences
- Deterministic chunk IDs via SHA256 hashing
- Chunk types: schema, rows, summary, pdf_page, pdf_section, website, golden_qa

web_scraper.py (10.4 KB)

Website crawler:
- Crawls sparrowi.com using httpx + BeautifulSoup
- Extracts text content + image metadata (alt text, figcaptions)
- Produces Chunk objects for vector ingestion
- Configurable: max 50 pages, 30s timeout per page

website_knowledge.py (15 KB)

Pre-built hardcoded knowledge chunks:
- Company overview, services (5 core services), team structure
- Office locations (India, UAE, Oman, Africa)
- Client lists by zone (North, West, South)
- Contact information, social media links
- Auto-ingested into Qdrant on every startup as baseline knowledge

database.py (42 KB)

SQLite database layer with tables for:
- leads: Customer leads with scoring (name, email, phone, score, status)
- conversations: Full chat history per session
- analytics_events: Event tracking (chat_opened, message_sent, etc.)
- feedback: User thumbs up/down votes on responses
- golden_qa: Curated Q&A pairs for high-quality answers
- email_contacts, email_templates, email_campaigns, email_sequences
- issues: Bug/issue tracking

lead_capture.py (22 KB)

Sales lead detection engine:
- Regex extraction of emails and phone numbers from messages
- Lead scoring: hot (70+), qualified (40+), new
- Intent classification for purchase signals
- Supports international phone formats (India, UAE, UK, US, Oman)

session_store.py (5.6 KB)

Conversation session management:
- Tries Redis first (host:6379), falls back to in-memory OrderedDict
- Stores conversation history per session_id
- Stores feedback (upvote/downvote) per message
- TTL: 1 hour per session

analytics.py (2.5 KB)

Event tracking wrapper:
- track_chat_opened(), track_message_sent(), track_suggestion_clicked()
- Writes events to SQLite via database.py

notifications.py (6.7 KB)

Email alert system:
- Sends SMTP email alerts when hot leads are captured
- Runs in background thread to not block API requests

sql_parser.py (5.6 KB)

SQL dump file parser:
- Extracts CREATE TABLE schemas (column names, types, constraints)
- Extracts INSERT INTO row data
- Returns structured ParsedTable objects

pdf_parser.py (5.8 KB)

PDF text extraction:
- Primary: pdfplumber for high-quality text
- Fallback: PyPDF2 for simpler PDFs
- OCR: pytesseract + PyMuPDF for image-heavy pages
- Text cleaning to fix extraction artifacts

models.py (9.6 KB)

Pydantic models for all API request/response schemas:
ChatRequest, ChatResponse, LeadResponse, EmailCampaignCreate, etc.

email/ directory

Full email marketing module:
- contacts.py: Email contact CRUD
- templates.py: Jinja2 email templates with variable substitution
- campaigns.py: Campaign management (create, schedule, send)
- sequences.py: Multi-step drip email sequences
- sender.py: Email sending via SMTP or Microsoft Graph API
- tracking.py: Open/click tracking via pixel and redirect URLs
- analytics_db.py: Email performance analytics
- scheduler.py: APScheduler for timed sends

4. Frontend Directory (frontend/)

The chatbot widget UI that gets embedded into the website pages.

chatbot-widget.html (6.7 KB)	Embeddable HTML snippet containing the chat widget markup. Gets injected by main.py before </body> in every scraped website page served by the backend.
app.js (38 KB)	Full chatbot UI logic: message rendering, session management, typing indicators, suggestion chips, lead capture form display, auto-scroll, and API communication with /chat endpoint.
chatbot-widget.css (27 KB)	Complete widget styling - chat bubble, message bubbles, input area, suggestion chips, lead forms, animations, responsive design.
index.html (6.9 KB)	Standalone chatbot page (for testing outside the website mirror).
style.css (22 KB)	Standalone page styling.
logo.png, sparrow-fab.png	Sparrow Interactive branding assets used in the widget.

5. Admin Dashboard (admin-dashboard/)

React single-page application built with Vite + TailwindCSS + shadcn/ui. Built output lives in admin-dashboard/dist/ and is served by FastAPI at /admin.

Page	Purpose
Dashboard.jsx	Overview stats: total leads, conversations, messages, conversion rates
Leads.jsx / LeadView.jsx	Lead management: view list, individual lead details, scoring, status updates, CSV export
Conversations.jsx	Chat history viewer with full message threads per session
Analytics.jsx	Charts and metrics: daily messages, top topics, response times
GoldenQA.jsx	Manage curated Q&A pairs that get ingested as high-priority chunks into Qdrant
Issues.jsx	Bug/issue tracker for chatbot problems
Settings.jsx	Admin configuration panel
email/ pages	Email campaign management: contacts, templates, campaigns, sequences, analytics
whatsapp/ pages	WhatsApp integration status and management

6. WhatsApp Bridge (whatsapp-bridge/)

Node.js service using @whiskeysockets/baileys (unofficial WhatsApp Web API). Runs on port 8004. Connects to WhatsApp via QR code scan, then acts as a bridge:

```
WhatsApp User sends message
        |
        v
whatsapp-bridge/server.js (port 8004)
        |
        | POST /chat with message + session_id
        v
FastAPI backend (port 8003)
        |
        | RAG pipeline -> Sarvam AI -> response
        v
whatsapp-bridge strips markdown, sends plain text back to WhatsApp
```

server.js (6.2 KB)	Main bridge: QR connection, message handler, markdown stripping, health endpoint
auth_info/	WhatsApp session credentials persisted across restarts
node_modules/	baileys, qrcode-terminal, pino dependencies

7. Data Directory (data/)

sparrowi.db (17 MB)

SQLite database storing ALL persistent data:
- leads table: captured leads with name, email, phone, score, status
- conversations table: every chat message (role, content, timestamp)
- analytics_events: user behavior tracking
- feedback: upvote/downvote on bot responses
- golden_qa: curated Q&A pairs for reliable answers
- email_contacts, email_templates, email_campaigns, email_sequences
- issues: reported problems

sample.sql (75 bytes)

Example SQL dump file for testing the ingestion pipeline

8. Qdrant Vector Database

Binary: qdrant.exe (70 MB)

Standalone Qdrant binary that runs locally on port 6333. No Docker required. Stores vectors for semantic similarity search.

Storage Directory: storage/

```
storage/
|-- raft_state.json     Qdrant cluster state (single-node, term=0)
|-- aliases/data.json   Collection aliases (empty: {})
+-- collections/        EMPTY - no persistent collection data
```

Why Qdrant Shows 'Deactivated' / Empty Collections

The storage/collections/ directory is EMPTY. This is the key finding:

Qdrant's on-disk persistence has NO saved collection data. When you previously ran qdrant.exe and saw 92 chunks, those were loaded IN-MEMORY during that session only. Once qdrant.exe was stopped, all vector data was lost.

This is BY DESIGN in this project. The application re-ingests all data on every startup:
1. website_knowledge.py hardcoded chunks (company info)
2. Golden QA pairs from SQLite database
3. Scraped website content from sparrowi.com

So the chunks are rebuilt fresh each time the app starts. The storage directory being empty simply means Qdrant is not currently running. When you start qdrant.exe + run.py, all 92 chunks will be recreated automatically.

Collection Details: sql_chunks

Property	Value
Collection Name	sql_chunks
Vector Dimensions	384 (all-MiniLM-L6-v2 output)
Distance Metric	Cosine Similarity
Total Points	92 chunks
Status (when running)	green

Chunk Breakdown by Source (92 Total)

Website Chunks (sparrowi.com) - ~68 chunks

Scraped from the live website. Includes:
- Homepage, About Us, Contact Us, Blog pages
- Service pages: Interactive Solutions, Immersive Experiences, Scale Models, Renders, Walkthroughs, Experiential Journeys, Architectural Visualization
- Portfolio items: Mahindra Experience Centre, Ambuja Neotia Brand Centre, etc.
- Blog posts: Jio World Convention Centre, Serum Institute collaboration, Sultan Haitham City
- Zone client pages: North Zone (DLF, Ace, Gaurs), West Zone (Runwal, Paradise)

Golden QA Chunks - ~12 chunks

Pre-curated high-quality Q&A pairs stored in SQLite, ingested as chunks:
- Company background and history

- Services offered (5 core services)
- Pricing and engagement process
- Team structure and key personnel (Rashmi Kohli, Ashish Mahajan, Tarrun Kumaria, etc.)
- Client relationship management
- Project execution workflow
- Contact information and office visits
- Portfolio overview (2,500+ projects)

PDF Policy Chunks - ~12 chunks

Extracted from 'Updated Policy Document - Sparrow Interactive (2025).pdf':
- Work hours and attendance (Mon-Sat, 0930-1830)
- Leave policies: Privilege Leave, Casual Leave, Medical Leave
- Compensatory offs for overtime work
- Late coming penalties
- Dinner/conveyance reimbursement for late work (after 2100 hrs)
- Leave application procedures

9. System Architecture & Data Flow

Startup Sequence

```
1. Start qdrant.exe            -> Qdrant listens on port 6333
2. Run: python run.py          -> Checks Qdrant health
3. FastAPI startup:            -> Launches uvicorn on port 8003
   a. Init SQLite tables       -> Creates leads, conversations, etc.
   b. Create Qdrant collection -> 'sql_chunks' (384-dim, cosine)
   c. Ingest website_knowledge -> Hardcoded company info chunks
   d. Ingest golden QA         -> Curated Q&A from SQLite
   e. Ingest scraped website   -> Crawl sparrowi.com pages
   f. Embed all chunks         -> all-MiniLM-L6-v2 (384-dim vectors)
   g. Upsert to Qdrant         -> 92 vectors stored
   h. Warm keyword cache       -> Pre-load all points for keyword search
4. (Optional) Start WhatsApp   -> node server.js on port 8004
```

Chat Request Flow

```
User Message (widget or WhatsApp)
        |
        v
POST /chat {query, session_id}
        |
        v
[1] Check greetings/small-talk -> instant response (no API)
        |
        v
[2] Check response cache -> return cached if hit
        |
        v
[3] Embed query with all-MiniLM-L6-v2 (384-dim vector)
        |
        v
[4] Parallel search:
    - Vector search (Qdrant cosine, top_k=6, min_score=0.35)
    - Keyword search (IDF-weighted, cached points)
        |
        v
[5] Merge + deduplicate results
        |
        v
[6] Re-rank with cross-encoder (ms-marco-MiniLM-L-6-v2)
    Filter below -5.0 score, keep top 3
        |
        v
[7] Build context (max 3000 chars from top chunks)
        |
        v
[8] Call Sarvam AI (sarvam-m model)
    System prompt + context + conversation history (last 4 turns)
    Timeout: 15s for answers, 8s for decisions
        |
        v
```

```
[9] Post-processing:
    - Detect lead intent, extract contact info
    - Score lead (hot/qualified/new)
    - Generate follow-up suggestions
    - Store in session + SQLite
    - Send email alert if hot lead
        |
        v
Return: {answer, sources, suggestions, lead_nudge}
```

10. Configuration (.env)

All configuration is in rag-chatbot/.env, loaded via Pydantic BaseSettings:

Variable	Value / Description
SARVAM_API_KEY	Sarvam AI API key for LLM calls
SARVAM_MODEL	sarvam-m (the LLM model used)
QDRANT_HOST	localhost
QDRANT_PORT	6333
QDRANT_COLLECTION	sql_chunks
EMBEDDING_MODEL	all-MiniLM-L6-v2
HOST / PORT	0.0.0.0:8003 (FastAPI server)
EMAIL_PROVIDER	microsoft (Microsoft Graph API)
MS_TENANT_ID	Azure AD tenant for email sending
MS_CLIENT_ID	Azure app registration client ID
MS_CLIENT_SECRET	Azure app secret (for Graph API auth)
MS_SENDER_EMAIL	rishi.sharma@sparrowi.com
SMTP_HOST/PORT/USER	Gmail SMTP (currently disabled)
EMAIL_TRACKING_BASE_URL	http://192.168.1.31:8003 (for open/click tracking)

11. Python Dependencies (requirements.txt)

Package	Purpose
fastapi==0.115.6	Web framework for all API endpoints
uvicorn[standard]==0.34.0	ASGI server to run FastAPI
sarvamai	Sarvam AI SDK for LLM chat completions
sentence-transformers==3.3.1	Embedding model (MiniLM) and cross-encoder (re-ranker)
qdrant-client==1.12.1	Qdrant vector database client
pydantic==2.10.4	Data validation and settings management
httpx==0.28.1	HTTP client for web scraping and API calls
beautifulsoup4==4.12.3	HTML parsing for web scraper
PyPDF2==3.0.1	PDF text extraction (fallback)
pdfplumber==0.11.9	PDF text extraction (primary)
PyMuPDF==1.25.3	PDF rendering for OCR
pytesseract==0.3.13	OCR for image-heavy PDF pages
Pillow==11.1.0	Image processing for OCR
redis==5.2.1	Redis client for session storage
APScheduler>=3.10.0	Task scheduling for email campaigns
msal>=1.28.0	Microsoft Authentication Library for Graph API
Jinja2>=3.1.0	Email template rendering
dnspython>=2.6.0	DNS validation for email contacts
dkimpy>=1.1.0	DKIM email authentication

12. Disk Size Breakdown

Component	Size
venv/ (Python packages)	962 MB
admin-dashboard/	140 MB
qdrant.exe	70 MB
whatsapp-bridge/	54 MB
Chatbot/ (website mirror)	238 MB
data/sparrowi.db	17 MB
backend/	~1 MB
frontend/	124 KB
storage/ (Qdrant)	2 KB
Test files	~1 MB

Total Project Size: 1.5 GB

Note: 96% of the disk space (1.44 GB) is consumed by dependencies (venv, node_modules, qdrant.exe, website mirror). The actual application source code is only ~2 MB.

13. Test Files

Multiple test suites for quality assurance:

test_chatbot.py	Core chatbot functionality tests
test_golden_qa.py	Golden QA accuracy tests
test_comprehensive_v2.py	Comprehensive V2 test suite
test_deep_inspection.py	Deep inspection of responses
test_complex.py / test_extreme.py	Edge case and complex query tests
stress_test_100.py / **stress_test_hard.py**	Load and stress testing
test_sales_bot.py / **test_sales_layer_1000.py**	Sales bot layer testing
run_100_queries.py	Batch query runner for benchmarking
test_100_conversations.py / **test_200_sessions.py**	Multi-session conversation tests

14. Summary

The Sparrowi RAG Chatbot is a full-stack AI sales assistant with the following architecture:

BACKEND: FastAPI Python server handling chat, ingestion, scraping, lead management, email campaigns, and serving the website mirror with embedded chatbot widget.

AI PIPELINE: User queries are embedded (MiniLM), searched in Qdrant (vector + keyword), re-ranked (cross-encoder), and answered by Sarvam AI LLM with retrieved context.

KNOWLEDGE BASE: 92 chunks in Qdrant from 3 sources: scraped website (~68), curated Q&A (~12), and company policy PDF (~12). All re-ingested on every startup.

STORAGE: SQLite for leads/conversations/analytics/email. Redis (optional) for sessions. Qdrant for vectors (in-memory, rebuilt on startup).

FRONTENDS: Embeddable chat widget (vanilla JS), React admin dashboard, WhatsApp bridge (Node.js).

INTEGRATIONS: Sarvam AI (LLM), Microsoft Graph API (email), WhatsApp (baileys), Tesseract (OCR), SMTP (Gmail backup).